

Data Quality Assessment

1. Table Validation

2. Missing Value Analysis

3. Duplicate Analysis

4. Referential Integrity Validation

5. Domain & Business Rule Validation

6. Date Validation

7. Summary of Data Quality Issues

STEP 1: Table Validation

In [1]: `import duckdb`

```
con = duckdb.connect("olist.duckdb")
```

In [2]: `con.sql("SHOW TABLES").show()`

name varchar
customers
geolocation
order_items
orders
payments
products
reviews
sellers
translation

In [3]: `con.sql("""
SELECT 'customers' AS table_name, COUNT(*) AS row_count
FROM customers`

```

UNION ALL

SELECT 'orders', COUNT(*) FROM orders

UNION ALL

SELECT 'order_items', COUNT(*) FROM order_items

UNION ALL

SELECT 'products', COUNT(*) FROM products

UNION ALL

SELECT 'sellers', COUNT(*) FROM sellers

UNION ALL

SELECT 'payments', COUNT(*) FROM payments

UNION ALL

SELECT 'reviews', COUNT(*) FROM reviews

UNION ALL

SELECT 'translation', COUNT(*) FROM translation

UNION ALL

SELECT 'geolocation', COUNT(*) FROM geolocation

""").show()

```

table_name varchar	row_count int64
customers	99441
orders	99441
order_items	112650
products	32951
sellers	3095
payments	103886
reviews	99224
translation	71
geolocation	1000163

Table Validation Verified all tables are loaded into DuckDB successfully. Observation:

		table_name	row_count	varchar	int64
products	32951	sellers	3095	payments	103886
		reviews	99224		

Step 2: Validate Primary Keys

```
In [10]: pk_checks = {
    "customers": "customer_id",
    "orders": "order_id",
    "products": "product_id",
    "sellers": "seller_id",
    "reviews": "review_id",
    "translation": "product_category_name"
}

for table, pk in pk_checks.items():

    query = f"""
    SELECT
        '{table}' AS table_name,
        COUNT(*) AS total_rows,
        COUNT(DISTINCT {pk}) AS unique_values
    FROM {table}
    """

    print(f"\nChecking {table}...")
    con.sql(query).show()
```

Checking customers...

table_name varchar	total_rows int64	unique_values int64
customers	99441	99441

Checking orders...

table_name varchar	total_rows int64	unique_values int64
orders	99441	99441

Checking products...

table_name varchar	total_rows int64	unique_values int64
products	32951	32951

Checking sellers...

table_name varchar	total_rows int64	unique_values int64
sellers	3095	3095

Checking reviews...

table_name varchar	total_rows int64	unique_values int64
reviews	99224	98410

Checking translation...

table_name varchar	total_rows int64	unique_values int64
translation	71	71

```
In [11]: pk_checks = {  
         "customers": "customer_id",
```

```

        "orders": "order_id",
        "products": "product_id",
        "sellers": "seller_id",
        "reviews": "review_id",
        "translation": "product_category_name"
    }

    results = []

    for table, pk in pk_checks.items():

        query = f"""
        SELECT
            COUNT(*) AS total_rows,
            COUNT(DISTINCT {pk}) AS unique_values,
            COUNT(*) - COUNT({pk}) AS null_values
        FROM {table}
        """

        df = con.sql(query).df()

        total_rows = df.iloc[0]["total_rows"]
        unique_values = df.iloc[0]["unique_values"]
        null_values = df.iloc[0]["null_values"]

        results.append({
            "table_name": table,
            "pk_column": pk,
            "total_rows": total_rows,
            "unique_values": unique_values,
            "null_values": null_values,
            "pk_valid": (
                total_rows == unique_values and
                null_values == 0
            )
        })

    import pandas as pd

    pd.DataFrame(results)

```

Out[11]:

	table_name	pk_column	total_rows	unique_values	null_values	pk_valid
0	customers	customer_id	99441	99441	0	True
1	orders	order_id	99441	99441	0	True
2	products	product_id	32951	32951	0	True
3	sellers	seller_id	3095	3095	0	True
4	reviews	review_id	99224	98410	0	False
5	translation	product_category_name	71	71	0	True

```
In [6]: con.sql("""
SELECT
    COUNT(*) AS total_rows,
    COUNT(DISTINCT review_id) AS unique_review_ids
FROM reviews
""").show()
```

total_rows int64	unique_review_ids int64
99224	98410

```
In [7]: con.sql("""
SELECT
    COUNT(*) AS null_review_ids
FROM reviews
WHERE review_id IS NULL
""").show()
```

null_review_ids int64
0

```
In [8]: con.sql("""
SELECT
    review_id,
    COUNT(*) AS occurrence_count
FROM reviews
GROUP BY review_id
HAVING COUNT(*) > 1
ORDER BY occurrence_count DESC
LIMIT 10
""").show()
```

review_id varchar	occurrence_count int64
38821b5c496b678cf91acc34892805ad	3
70509c441d994fa03d6c1457930c9024	3
2d6ac45f859465b5c185274a1c929637	3
832acec9bbf4efe65c3fb6423d8b4ed7	3
ddc52555ca27b0fe67d5255147682d2d	3
9e25d6e3025e9b9a0fc7f03588d33e2b	3
f4bb9d6dd4fb6dcc2298f0e7b17b8e1e	3
44e9f871226d8a130de3fc39dfbdf0c5	3
69a1068c3128a14994e3e422e4539e04	3
dbdf1ea31790c8ecfcc6750525661a9b	3

10 rows

2 columns

Validate Order Items Composite PK

```
In [17]: composite_pk_checks = [
    ("order_items", ["order_id", "order_item_id"]),
    ("payments", ["order_id", "payment_sequential"])
]

results = []

for table, cols in composite_pk_checks:

    concat_cols = " || '-' || ".join(cols)

    query = f"""
    SELECT
        COUNT(*) AS total_rows,
        COUNT(DISTINCT ({concat_cols})) AS unique_combinations
    FROM {table}
    """

    df = con.sql(query).df()

    total_rows = df.iloc[0]["total_rows"]
    unique_combinations = df.iloc[0]["unique_combinations"]

    results.append({
        "table_name": table,
        "composite_key": " || ".join(cols),
        "total_rows": total_rows,
        "unique_combinations": unique_combinations,
        "pk_valid": total_rows == unique_combinations
    })

import pandas as pd

pd.DataFrame(results)
```

Out[17]:	table_name	composite_key	total_rows	unique_combinations	pk_valid
0	order_items	order_id, order_item_id	112650	112650	True
1	payments	order_id, payment_sequential	103886	103886	True

```
In [18]: con.sql("""
SELECT
    COUNT(*) - COUNT(order_id) AS order_id_nulls,
    COUNT(*) - COUNT(order_item_id) AS order_item_id_nulls
FROM order_items
""").show()
```

order_id_nulls	order_item_id_nulls
int64	int64
0	0

```
In [20]: con.sql("""
SELECT
    COUNT(*) - COUNT(order_id) AS order_id_nulls,
    COUNT(*) - COUNT(payment_sequential) AS payment_sequential_nulls
FROM payments
""").show()
```

order_id_nulls	payment_sequential_nulls
int64	int64
0	0

Composite PK order_items → (order_id, order_item_id) payments → (order_id, payment_sequential)

Step 2: Foreign Key Validation (Referential Integrity)

```
In [12]: fk_checks = [
    ("orders", "customer_id", "customers", "customer_id"),

    ("order_items", "order_id", "orders", "order_id"),
    ("order_items", "product_id", "products", "product_id"),
    ("order_items", "seller_id", "sellers", "seller_id"),

    ("payments", "order_id", "orders", "order_id"),

    ("reviews", "order_id", "orders", "order_id")
]
```

```
In [15]: results = []

for child_table, child_col, parent_table, parent_col in fk_checks:
```

```

query = f"""
SELECT
    COUNT(*) AS parent_rows,
    COUNT(DISTINCT {parent_col}) AS parent_unique_values,

    (
        SELECT COUNT(*)
        FROM {child_table} c
        LEFT JOIN {parent_table} p
            ON c.{child_col} = p.{parent_col}
        WHERE p.{parent_col} IS NULL
    ) AS orphan_records

FROM {parent_table}
"""

df = con.sql(query).df()

parent_rows = df.iloc[0]["parent_rows"]
parent_unique_values = df.iloc[0]["parent_unique_values"]
orphan_records = df.iloc[0]["orphan_records"]

parent_unique = parent_rows == parent_unique_values

results.append({
    "child_table": child_table,
    "child_column": child_col,
    "parent_table": parent_table,
    "parent_column": parent_col,
    "parent_unique": parent_unique,
    "orphan_records": orphan_records,
    "fk_valid": parent_unique and orphan_records == 0
})

import pandas as pd

pd.DataFrame(results)

```

Out[15]:

	child_table	child_column	parent_table	parent_column	parent_unique	orphan_records
0	orders	customer_id	customers	customer_id	True	0
1	order_items	order_id	orders	order_id	True	0
2	order_items	product_id	products	product_id	True	0
3	order_items	seller_id	sellers	seller_id	True	0
4	payments	order_id	orders	order_id	True	0
5	reviews	order_id	orders	order_id	True	0

Valid Foreign Keys orders.customer_id ↓ customers.customer_id order_items.order_id ↓ orders.order_id order_items.product_id ↓ products.product_id order_items.seller_id ↓ sellers.seller_id payments.order_id ↓ orders.order_id reviews.order_id ↓ orders.order_id candidate FK - translation.product_category_name

Missing Value Analysis

```
In [22]: con.sql("""
DESCRIBE products
""").show()
```

column_name varchar	column_type varchar	null varchar	key varchar	default varchar	extra varchar
product_id	VARCHAR	YES	NULL	NULL	NULL
product_category_name	VARCHAR	YES	NULL	NULL	NULL
product_name_lenght	BIGINT	YES	NULL	NULL	NULL
product_description_lenght	BIGINT	YES	NULL	NULL	NULL
product_photos_qty	BIGINT	YES	NULL	NULL	NULL
product_weight_g	BIGINT	YES	NULL	NULL	NULL
product_length_cm	BIGINT	YES	NULL	NULL	NULL
product_height_cm	BIGINT	YES	NULL	NULL	NULL
product_width_cm	BIGINT	YES	NULL	NULL	NULL

```
In [24]: con.sql("""
SELECT
    COUNT(*) AS total_rows,
    COUNT(*) - COUNT(product_category_name) AS null_rows,
    ROUND(
        100.0 *
        (COUNT(*) - COUNT(product_category_name))
        / COUNT(*),
        2
    ) AS null_percentage
FROM products
""").show()
```

total_rows int64	null_rows int64	null_percentage double
32951	610	1.85

```
In [25]: product_columns = [
    "product_id",
    "product_category_name",
    "product_name_lenght",
    "product_description_lenght",
    "product_photos_qty",
    "product_weight_g",
    "product_length_cm",
    "product_height_cm",
    "product_width_cm"
]
```

```
In [29]: product_columns = [
    "product_id",
    "product_category_name",
    "product_name_lenght",
    "product_description_lenght",
    "product_photos_qty",
    "product_weight_g",
    "product_length_cm",
    "product_height_cm",
    "product_width_cm"
]

results = []

for col in product_columns:

    query = f"""
    SELECT
        COUNT(*) - COUNT({col}) AS null_count,
        ROUND(
            100.0 * (COUNT(*) - COUNT({col})) / COUNT(*),
            2
        ) AS null_percentage
    FROM products
    """

    df = con.sql(query).df()

    results.append({
        "column_name": col,
        "null_count": df.iloc[0]["null_count"],
        "null_percentage": df.iloc[0]["null_percentage"]
    })

import pandas as pd

pd.DataFrame(results).sort_values(
    by="null_percentage",
    ascending=False
)
```

Out[29]:

	column_name	null_count	null_percentage
1	product_category_name	610.0	1.85
3	product_description_lenght	610.0	1.85
2	product_name_lenght	610.0	1.85
4	product_photos_qty	610.0	1.85
5	product_weight_g	2.0	0.01
7	product_height_cm	2.0	0.01
6	product_length_cm	2.0	0.01
8	product_width_cm	2.0	0.01
0	product_id	0.0	0.00

Products Table - Missing Value Analysis

Findings:

1. product_category_name, product_name_lenght, product_description_lenght and product_photos_qty contain 610 missing values (1.85%).
2. The identical count across these columns suggests that the same set of products have incomplete metadata records.
3. Product dimensions and weight attributes contain only 2 missing values each (0.01%).
4. product_id contains no missing values, supporting its role as a valid Primary Key.

Business Impact:

Missing category and product-description information may affect category-level and product-level analysis. However, the overall percentage of missing records is low.

In [30]:

```
con.sql("""  
SHOW TABLES  
""").df()
```

Out[30]:

	name
0	customers
1	geolocation
2	order_items
3	orders
4	payments
5	products
6	reviews
7	sellers
8	translation

```
In [31]: tables = con.sql("""
SHOW TABLES
""").df()["name"].tolist()

print(tables)
```

```
['customers', 'geolocation', 'order_items', 'orders', 'payments', 'products', 'reviews', 'sellers', 'translation']
```

```
In [33]: results = []

tables = [
    "customers",
    "orders",
    "order_items",
    "products",
    "sellers",
    "payments",
    "reviews",
    "translation"
]

for table in tables:

    columns = con.sql(f"""
DESCRIBE {table}
""").df()["column_name"].tolist()

    for col in columns:

        query = f"""
SELECT
    COUNT(*) - COUNT({col}) AS null_count,
    ROUND(
        100.0 *
        (COUNT(*) - COUNT({col}))
        / COUNT(*),
```

```

        2
    ) AS null_percentage
FROM {table}
"""

df = con.sql(query).df()

results.append({
    "table_name": table,
    "column_name": col,
    "null_count": df.iloc[0]["null_count"],
    "null_percentage": df.iloc[0]["null_percentage"]
})

```

```

In [34]: import pandas as pd

missing_report = pd.DataFrame(results)

missing_report.sort_values(
    by="null_percentage",
    ascending=False
)

```

Out[34]:

	table_name	column_name	null_count	null_percentage
41	reviews	review_comment_title	87656.0	88.34
42	reviews	review_comment_message	58247.0	58.70
11	orders	order_delivered_customer_date	2965.0	2.98
22	products	product_name_lenght	610.0	1.85
21	products	product_category_name	610.0	1.85
23	products	product_description_lenght	610.0	1.85
24	products	product_photos_qty	610.0	1.85
10	orders	order_delivered_carrier_date	1783.0	1.79
9	orders	order_approved_at	160.0	0.16
25	products	product_weight_g	2.0	0.01
26	products	product_length_cm	2.0	0.01
27	products	product_height_cm	2.0	0.01
28	products	product_width_cm	2.0	0.01
3	customers	customer_city	0.0	0.00
0	customers	customer_id	0.0	0.00
14	order_items	order_item_id	0.0	0.00
13	order_items	order_id	0.0	0.00
12	orders	order_estimated_delivery_date	0.0	0.00
7	orders	order_status	0.0	0.00
8	orders	order_purchase_timestamp	0.0	0.00
4	customers	customer_state	0.0	0.00
5	orders	order_id	0.0	0.00
6	orders	customer_id	0.0	0.00
2	customers	customer_zip_code_prefix	0.0	0.00
1	customers	customer_unique_id	0.0	0.00
19	order_items	freight_value	0.0	0.00
20	products	product_id	0.0	0.00
16	order_items	seller_id	0.0	0.00
15	order_items	product_id	0.0	0.00
17	order_items	shipping_limit_date	0.0	0.00

	table_name	column_name	null_count	null_percentage
18	order_items	price	0.0	0.00
30	sellers	seller_zip_code_prefix	0.0	0.00
29	sellers	seller_id	0.0	0.00
31	sellers	seller_city	0.0	0.00
32	sellers	seller_state	0.0	0.00
35	payments	payment_type	0.0	0.00
36	payments	payment_installments	0.0	0.00
33	payments	order_id	0.0	0.00
34	payments	payment_sequential	0.0	0.00
38	reviews	review_id	0.0	0.00
37	payments	payment_value	0.0	0.00
40	reviews	review_score	0.0	0.00
39	reviews	order_id	0.0	0.00
43	reviews	review_creation_date	0.0	0.00
44	reviews	review_answer_timestamp	0.0	0.00
45	translation	product_category_name	0.0	0.00
46	translation	product_category_name_english	0.0	0.00

```
In [35]: missing_report[
    missing_report["null_count"] > 0
].sort_values(
    by="null_percentage",
    ascending=False
)
```

Out[35]:

	table_name	column_name	null_count	null_percentage
41	reviews	review_comment_title	87656.0	88.34
42	reviews	review_comment_message	58247.0	58.70
11	orders	order_delivered_customer_date	2965.0	2.98
22	products	product_name_lenght	610.0	1.85
21	products	product_category_name	610.0	1.85
23	products	product_description_lenght	610.0	1.85
24	products	product_photos_qty	610.0	1.85
10	orders	order_delivered_carrier_date	1783.0	1.79
9	orders	order_approved_at	160.0	0.16
26	products	product_length_cm	2.0	0.01
25	products	product_weight_g	2.0	0.01
28	products	product_width_cm	2.0	0.01
27	products	product_height_cm	2.0	0.01

In [36]:

```
con.sql("""
SELECT
    order_status,
    COUNT(*) AS order_count
FROM orders
WHERE order_delivered_customer_date IS NULL
GROUP BY order_status
ORDER BY order_count DESC
""").show()
```

order_status varchar	order_count int64
shipped	1107
canceled	619
unavailable	609
invoiced	314
processing	301
delivered	8
created	5
approved	2

Missing Value Analysis - Key Insights

Reviews Table

- `review_comment_title` contains 88.34% missing values.
- `review_comment_message` contains 58.70% missing values.
- These missing values are likely expected as many customers provided only review ratings without textual feedback.

Orders Table

- `order_delivered_customer_date` contains 2.98% missing values.
- `order_delivered_carrier_date` contains 1.79% missing values.
- `order_approved_at` contains 0.16% missing values.
- Missing date values may correspond to cancelled, unavailable, or incomplete orders and require further investigation using `order_status`.

Products Table

- `product_category_name`, `product_name_lenght`, `product_description_lenght`, and `product_photos_qty` each contain 610 missing values (1.85%).
- The identical missing counts suggest the same set of products have incomplete metadata.
- Missing product metadata may impact category-based and product-level analyses.

Product Dimensions

- `product_weight_g`, `product_length_cm`, `product_height_cm`, and `product_width_cm` contain only 2 missing records each (0.01%).
- These missing values are negligible and unlikely to affect analysis.

Overall Insights

- Most missing values appear to be business-process driven rather than major data quality issues.
- Product metadata is the primary area requiring attention before product/category analysis.
- No missing values were found in validated primary key columns.
- Referential integrity and key validations remain unaffected by the identified missing values.

1. Order Date Columns - Are these NULLs caused by order status?

```
In [46]: con.sql("""SELECT
            order_status,
            COUNT(*) AS order_count
        FROM orders
        WHERE order_delivered_customer_date IS NULL
```

```
GROUP BY order_status
ORDER BY order_count DESC;""").show()
```

order_status varchar	order_count int64
shipped	1107
canceled	619
unavailable	609
invoiced	314
processing	301
delivered	8
created	5
approved	2

Investigation: Missing Customer Delivery Date

Analysis of orders with NULL `order_delivered_customer_date` shows that most missing values are associated with:

- shipped
- canceled
- unavailable
- invoiced
- processing

These statuses represent orders that were not yet delivered or were never completed. Therefore, the missing delivery date is expected and does not indicate a data quality issue.

However, 8 orders have a status of `delivered` while still having a NULL delivery date.

This appears inconsistent with the expected business process and may represent a small data quality issue.

2. Product Metadata Columns - Are these the same 610 products?

```
In [48]: df = con.sql("""
SELECT *
FROM products
WHERE product_category_name IS NULL
LIMIT 10;
""").df()

display(df)
```

	product_id	product_category_name	product_name_lenght	produc
0	a41e356c76fab66334f36de622ecbd3a	None	<NA>	
1	d8dee61c2034d6d075997acef1870e9b	None	<NA>	
2	56139431d72cd51f19eb9f7dae4d1617	None	<NA>	
3	46b48281eb6d663ced748f324108c733	None	<NA>	
4	5fb61f482620cb672f5e586bb132eae9	None	<NA>	
5	e10758160da97891c2fdcabc35f0f031d	None	<NA>	
6	39e3b9b12cd0bf8ee681bbc1c130feb5	None	<NA>	
7	794de06c32a626a5692ff50e4985d36f	None	<NA>	
8	7af3e2da474486a3519b0cba9dea8ad9	None	<NA>	
9	629beb8e7317703dcc5f35b5463fd20e	None	<NA>	

Product Metadata Investigation

A deeper investigation of products with NULL `product_category_name` revealed that the same records are also missing:

- `product_name_lenght`
- `product_description_lenght`
- `product_photos_qty`

However, these products still contain valid logistics-related attributes such as:

- `product_weight_g`
- `product_length_cm`
- `product_height_cm`
- `product_width_cm`

This indicates that the affected products are not entirely missing records. Instead, they appear to have incomplete catalog metadata while retaining physical product specifications.

The issue impacts approximately 1.85% of products (610 records).`

```
In [49]: con.sql("""
SELECT COUNT(*) AS invalid_records
FROM orders
WHERE order_approved_at < order_purchase_timestamp
""").show()
```

invalid_records
int64
0

```
In [53]: con.sql("""
SELECT COUNT(*) AS invalid_records
FROM orders
WHERE order_delivered_carrier_date < order_approved_at
""").show()
```

invalid_records
int64
1359

```
In [61]: con.sql("""
SELECT
    order_id,
    order_purchase_timestamp,
    order_approved_at,
    order_delivered_carrier_date
FROM orders
WHERE order_delivered_carrier_date < order_approved_at
LIMIT 20;
""").df()
```

Out[61]:

	order_id	order_purchase_timestamp	order_approved_at	ord
0	dcb36b511fcac050b97cd5c05de84dc3	2018-06-07 19:03:12	2018-06-12 23:31:02	
1	688052146432ef8253587b930b01a06d	2018-04-22 08:48:13	2018-04-24 18:25:22	
2	58d4c4747ee059eeeb865b349b41f53a	2018-07-21 12:49:32	2018-07-26 23:31:53	
3	412fccb2b44a99b36714bca3fef8ad7b	2018-07-22 22:30:05	2018-07-23 12:31:53	
4	56a4ac10a4a8f2ba7693523bb439eede	2018-07-22 13:04:47	2018-07-27 23:31:09	
5	32e4fa9bb468884309b58b9348de70c3	2018-07-04 16:49:21	2018-07-05 16:33:06	
6	4df92d82d79c3b52c7138679fa9b07fc	2018-07-24 11:32:11	2018-07-29 23:30:52	
7	16e38caa92e342c7780f68832f832d4d	2018-05-07 01:09:09	2018-05-07 16:52:39	
8	b9afddbdcfadc9a87b41a83271c3e888	2018-08-16 13:50:48	2018-08-16 14:05:13	
9	6051e6d3da9a50b7325cbe9c81025062	2018-07-03 23:40:16	2018-07-05 16:31:26	
10	7ebe19943d85e81a7b0eb108681276b7	2018-07-22 10:28:14	2018-07-23 11:31:43	
11	95fa26b6ea476ed040e592e0a825d6f0	2018-06-06 13:48:42	2018-06-06 14:13:02	
12	c6512da3ccb5263223de71b8888d4db1	2018-07-04 11:38:50	2018-07-05 16:40:14	
13	cee9f1845c95efd83885a0b2f7a22b83	2018-07-29 12:47:00	2018-08-01 16:31:40	
14	ad133696906f6a78826daa0911b7daec	2018-06-15 15:41:22	2018-06-15 16:19:23	
15	0fe8aad910ff05e73b0ddb0a89e61b21	2018-07-01 20:11:26	2018-07-05 16:02:47	
16	59dba471cbd7e791c3bb0d83aa705c49	2018-07-04 17:15:18	2018-07-05 16:33:22	
17	74e033208dc13a7b8127eb8e73d09b76	2018-05-02 10:48:44	2018-05-02 11:13:45	
18	dda3e0dbde3a2cd09432650b3634582d	2018-04-22 20:13:58	2018-04-24 18:37:08	

	order_id	order_purchase_timestamp	order_approved_at	ord
19	a6b58794fd2ba533359a76c08df576e3	2018-05-14 15:18:23	2018-05-14 15:33:35	

```
In [54]: con.sql("""
SELECT COUNT(*) AS invalid_records
FROM orders
WHERE order_delivered_customer_date < order_delivered_carrier_date
""").show()
```

invalid_records	
int64	
	23

```
In [62]: con.sql("""
SELECT
    order_id,
    order_delivered_carrier_date,
    order_delivered_customer_date
FROM orders
WHERE order_delivered_customer_date < order_delivered_carrier_date
LIMIT 20;
""").df()
```

Out[62]:

	order_id	order_delivered_carrier_date	order_delivered_custon
0	a1abeb653a4d4cd1e142ccb8c82cd069	2017-07-28 16:57:58	2017-07-25
1	383aa8b2724fe452d9ccd9934a8c628b	2017-07-07 17:22:41	2017-07-06
2	cb1134f9010d242e9515ad1c78ec0c39	2017-07-20 19:22:02	2017-07-19
3	dceb62e8fa94b46006c9554fed743df0	2017-08-01 18:23:30	2017-07-26
4	5f9d46795c3126674e52becb3a1a517f	2017-07-20 23:03:42	2017-07-20
5	8c78d01de3a9009e23d6877a7cc9be20	2016-10-26 11:41:53	2016-10-25
6	b27af682321527a6349f1761eb3f360c	2017-06-27 14:51:54	2017-06-26
7	1cc3ae63cafff2d6c3ee3e78e074acf	2017-08-10 18:28:56	2017-08-10
8	e37f11cae9985ca58f0b56f268720537	2017-08-01 18:17:47	2017-07-31
9	fa3e37584f4fdb1ded0e0de700dfcb4e	2017-08-09 18:18:43	2017-08-01
10	c1e2bf2b7dd3309f2f5356c6b63968fa	2017-03-02 17:34:26	2017-02-14
11	b866af202be0692766081310cd4085e1	2017-02-20 02:32:08	2017-02-15
12	29941903985f944b0ffc49c479c1547d	2017-06-09 15:07:29	2017-06-02
13	76458889992169d3135b264dc13aec67	2016-10-26 11:43:06	2016-10-20
14	0922ee1619de7b995648e5a8407afb91	2017-07-14 12:41:34	2017-07-12
15	fbffbaaa30f4077e87ec6acdb835b278	2017-08-04 20:04:17	2017-08-03
16	3ee6694bb99d5a6db0deceffcd099597	2017-07-11 18:15:40	2017-07-11
17	19feb5627c41ea1b36a8e50a469b3644	2016-10-26 11:42:05	2016-10-20
18	d5558a097766363b8e76b38c43332e8a	2017-02-15 08:55:26	2017-02-10
19	771c4f1f521f462e4b95619e648aaeab	2017-03-30 13:14:34	2017-03-28

```
In [55]: con.sql("""
SELECT COUNT(*) AS invalid_records
FROM orders
WHERE order_delivered_customer_date < order_purchase_timestamp
""").show()
```

invalid_records
int64
0

```
In [56]: con.sql("""
SELECT COUNT(*) AS invalid_records
FROM orders
```

```
WHERE order_delivered_customer_date < order_purchase_timestamp  
""").show()
```

invalid_records
int64
0

```
In [57]: con.sql("""  
SELECT COUNT(*) AS negative_prices  
FROM order_items  
WHERE price < 0  
""").show()
```

negative_prices
int64
0

```
In [58]: con.sql("""  
SELECT COUNT(*) AS negative_freight  
FROM order_items  
WHERE freight_value < 0  
""").show()
```

negative_freight
int64
0

```
In [60]: con.sql("""  
SELECT COUNT(*) AS negative_payments  
FROM payments  
WHERE payment_value < 0  
""").show()
```

negative_payments
int64
0

Date Validation Findings

Rule 1

Purchase Date <= Approval Date

Result: 0 Invalid Records

Conclusion: Business rule satisfied.

Rule 2

Approval Date <= Carrier Date

Result: 1,359 Invalid Records

Observation: Carrier handover timestamp occurs before approval timestamp for a subset of orders. Further investigation required.

Rule 3

Carrier Date <= Customer Delivery Date

Result: 23 Invalid Records

Observation: Customer delivery timestamp occurs before carrier handover timestamp for a very small number of records. This appears to be a data-quality anomaly.

Rule 4

Purchase Date <= Customer Delivery Date

Result: 0 Invalid Records

Conclusion: Business rule satisfied.

Approval Date vs Carrier Date

A total of 1,359 orders have `order_delivered_carrier_date < order_approved_at` .

Investigation of sample records indicates that the difference is typically small, often ranging from a few minutes to a few days.

This suggests potential timestamp recording or process-ordering differences rather than a major data-quality issue.

No evidence was found that customer delivery occurred before order purchase.

Customer Delivery vs Carrier Delivery Validation

23 records were identified where `order_delivered_customer_date` occurred before `order_delivered_carrier_date`.

This sequence contradicts the expected fulfillment process, indicating a data-quality anomaly.

Although the issue affects only 23 records (approximately 0.02% of all orders), these records were flagged as date inconsistencies.

Date Validation Summary

1. Purchase Date <= Approval Date

- Passed
- 0 invalid records

2. Purchase Date <= Customer Delivery Date

- Passed
- 0 invalid records

3. Approval Date <= Carrier Date

- 1,359 records violated the rule.
- Investigation suggests timestamp sequencing differences rather than major data-quality issues.

4. Carrier Date <= Customer Delivery Date

- 23 records violated the rule.
- These represent genuine date anomalies.

```
In [64]: con.sql("""
SELECT COUNT(*) AS negative_prices
FROM order_items
WHERE price < 0
""").show()
```

negative_prices
int64
0

```
In [65]: con.sql("""
SELECT COUNT(*) AS negative_freight
FROM order_items
WHERE freight_value < 0
""").show()
```

negative_freight int64
0

```
In [66]: con.sql("""
SELECT COUNT(*) AS negative_payments
FROM payments
WHERE payment_value < 0
""").show()
```

negative_payments int64
0

Data Quality Assessment Summary

Primary Key Validation

Validated Primary Keys:

- customers.customer_id
- orders.order_id
- products.product_id
- sellers.seller_id
- translation.product_category_name

Validated Composite Primary Keys:

- order_items (order_id, order_item_id)
- payments (order_id, payment_sequential)

Observation:

- review_id is not unique and therefore cannot be considered a Primary Key.

Foreign Key Validation

Validated Relationships:

- orders.customer_id → customers.customer_id

- order_items.order_id → orders.order_id
- order_items.product_id → products.product_id
- order_items.seller_id → sellers.seller_id
- payments.order_id → orders.order_id
- reviews.order_id → orders.order_id

Observation:

- products.product_category_name has an intended lookup relationship with translation.product_category_name.
 - However, some product categories are missing from the translation table, resulting in referential integrity issues.
-

Missing Value Analysis

Reviews Table:

- review_comment_title contains 88.34% missing values.
- review_comment_message contains 58.70% missing values.

These missing values are expected because many customers provide ratings without textual feedback.

Orders Table:

- order_delivered_customer_date contains 2.98% missing values.
- order_delivered_carrier_date contains 1.79% missing values.
- order_approved_at contains 0.16% missing values.

Investigation showed that most missing delivery dates belong to orders with statuses such as:

- shipped
- canceled
- unavailable
- invoiced
- processing

Therefore, most date-related missing values are business-process driven and expected.

Products Table:

- product_category_name
- product_name_lenght
- product_description_lenght
- product_photos_qty

each contain 610 missing values (1.85%).

Investigation revealed that these products contain valid physical specifications but incomplete catalog metadata.

Date Validation

Validation Rules Checked:

1. Purchase Date $\leq$ Approval Date
 - Passed
 - 0 invalid records
 2. Purchase Date $\leq$ Customer Delivery Date
 - Passed
 - 0 invalid records
 3. Approval Date $\leq$ Carrier Date
 - 1,359 records violated the expected sequence.
 - Investigation suggests timestamp sequencing differences rather than major process issues.
 4. Carrier Date $\leq$ Customer Delivery Date
 - 23 records violated the rule.
 - These records represent genuine date anomalies.
-

Financial Validation

Checks Performed:

- Negative Product Prices
- Negative Freight Charges
- Negative Payment Values

Result:

- No negative values found.
-

Overall Assessment

The dataset demonstrates strong overall data quality.

Key observations include:

- Valid primary key structure across major entities.
- Strong referential integrity across core business tables.
- Expected missing values driven by business processes.
- Minor product metadata completeness issues.
- Small number of timestamp anomalies.
- No financial data inconsistencies.

The dataset is suitable for exploratory analysis and business intelligence reporting.

```
In [74]: # How many customers exist?
con.sql("""SELECT COUNT(customer_id) AS total_customers FROM customers;""").show()
```

total_customers
int64
99441

```
In [78]: # How many unique customers exist?
con.sql("""SELECT COUNT(DISTINCT customer_unique_id) AS unique_customers FROM cust
```

unique_customers
int64
96096

```
In [79]: # How many orders exist?
con.sql("""select count(*) as total_orders from orders;""")
```

```
Out[79]:
```

total_orders
int64
99441

```
In [80]: # How many products exist?
con.sql("""select count(*) as total_products from products;""").show()
```

total_products
int64
32951

```
In [82]: # How many sellers exist?
con.sql("""select count(*) as total_sellers from sellers;""").show()
```

total_sellers
int64
3095

```
In [ ]:
```

```
In [ ]:
```

```
In [71]: con.sql("""SELECT * FROM customers;""")
```

Out[71]:

customer_id		customer_unique_id		customer_z
ip_code_prefix	customer_city	customer_state		va
varchar	varchar	varchar		
06b8999e2fba1a1fbc88172c00ba8bc7		861eff4711a542e4b93843c6dd7febb0		14409
franca	SP			
18955e83d337fd6b2def6b18a428ac77		290c77bc529b7ac935b93aa66c333dc3		09790
sao bernardo do campo	SP			
4e7b3e00288586ebd08712fdd0374a03		060e732b5b29e8181a18229c7b0b2b5e		01151
sao paulo	SP			
b2b6027bc5c5109e529d4dc6358b12c3		259dac757896d24d7702b9acbbff3f3c		08775
mogi das cruzeiras	SP			
4f2d8ab171c80ec8364f7c12e35b23ad		345ecd01c38d18a9036ed96c73b8d066		13056
campinas	SP			
879864dab9bc3047522c92c82e1212b8		4c93744516667ad3b8f1fb645a3116a4		89254
jaragua do sul	SC			
fd826e7cf63160e536e0908c76c3f441		addec96d2e059c80c30fe6871d30d177		04534
sao paulo	SP			
5e274e7a0c3809e14aba7ad5aae0d407		57b2a98a409812fe9618067b6b8ebe4f		35182
timoteo	MG			
5adf08e34b2e993982a47070956c5c65		1175e95fb47ddff9de6b2b06188f7e0d		81560
curitiba	PR			
4b7139f34592b3a31687243a302fa75b		9afe194fb833f79e300e37e580171f22		30575
belo horizonte	MG			
.	.	.		.
.	.	.		.
.	.	.		.
.	.	.		.
7caae07d3441f77333b6e47ce323d251		b2c72d9d87482e5f6ba18d5a8c68a93d		24315
niteroi	RJ			
37b1637fce7efc4438e4e9345d781600		8c8599d9df59347ee1ff99500b68bd94		61905
maracanau	CE			
ec86a47ce0050ff9be984c455dc57231		333116644860dfd23cda707c267686b8		85550
coronel vivida	PR			
1a475ff9958a875936b3da7c7367212c		f9b5d3c8bd2dc9c26ff00bde73171554		45360
maracas	BA			
eeac04e8c3bbdb160700ca457bdda435		2dc3d7a1a8e362242853cfa233a8ee04		93950
dois irmaos	RS			
ae9dcc4124363f82adbd2148d6e240b7		4cc53a257e7ac8a51fb35a7be04553fb		22795
rio de janeiro	RJ			
dac2ba4d4844ffb03b2c8e4880a8405f		bb80ab8ffb7d3b9f9a11b3f06eef10c3		04115
sao paulo	SP			
457aa2d8f32998f155e8d037e5833c89		28a14fa019f261a7bddfc820dae8d616		30315
belo horizonte	MG			
3d64d81cb7d22325994c3ef4ad93d2e0		c5f6047fb345ffd234cf5b26268988be		18305
capao bonito	SP			
b240cbf2b50a10ee1f80b42c56b99069		c47ec91fad63554dd8fe3e33188faf7d		36026
juiz de fora	MG			

```
? rows (>9999 rows, 20 shown)
5 columns
```

```
In [83]: con.close()
```

```
In [ ]:
```